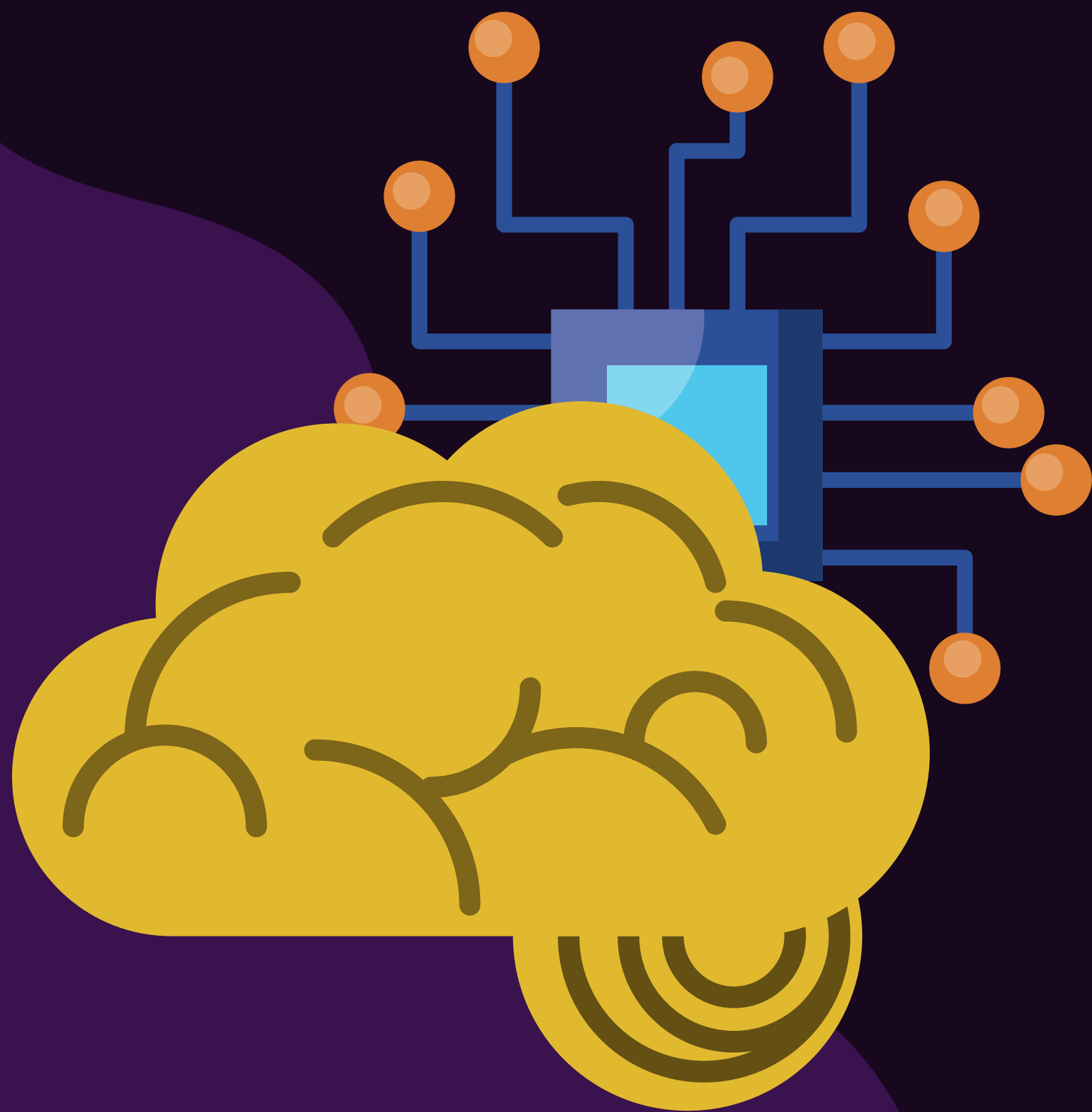




INTRODUCCIÓN A IMÁGENES Y REDES NEURONALES

Sebastian Muñoz → jumunozle@unal.edu.co

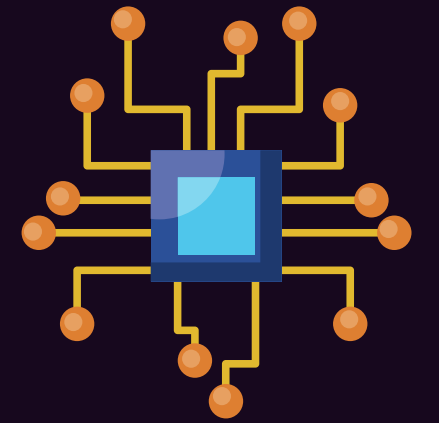
Carlos Camacho → cacamacho@unal.edu.co

Juan Daniel Ramírez → juaramriezmo@unal.edu.co

Sergio David Lopez → slopezpa@unal.edu.co

Miércoles, 11 de junio de 2025.

Contenidos generales



1 ¿Qué es una imagen para una red neuronal?

2 Preprocesamiento y normalización

3 Ejemplo visual de transformación

4 Dificultad de modelar imágenes con redes densas

5 Ventajas de representarlas como tensores

6 Aplicaciones típicas

7 Relación con el aprendizaje automático tradicional

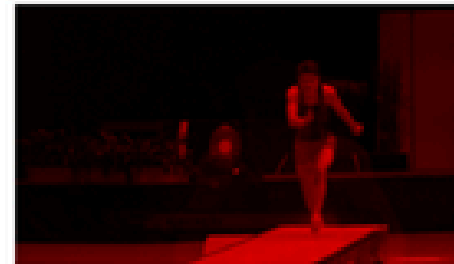
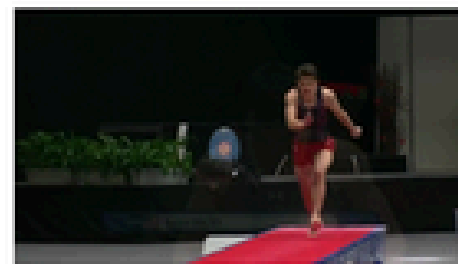
1. ¿Qué es una imagen para una red neuronal?

Imagen = Tensor tridimensional

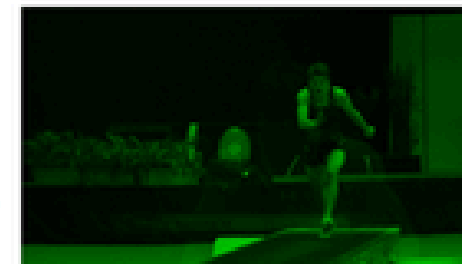
(Alto × Ancho × Canales)

Cada canal: matriz de intensidades

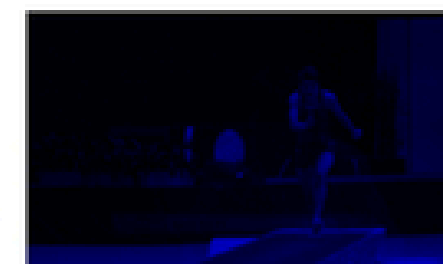
RGB channels
extraction



120	16	234	245	12
23	187	235	137	78
35	109	100	23	198
23	134	245	230	23
87	65	234	198	65
129	197	240	35	150
65	250	243	21	93

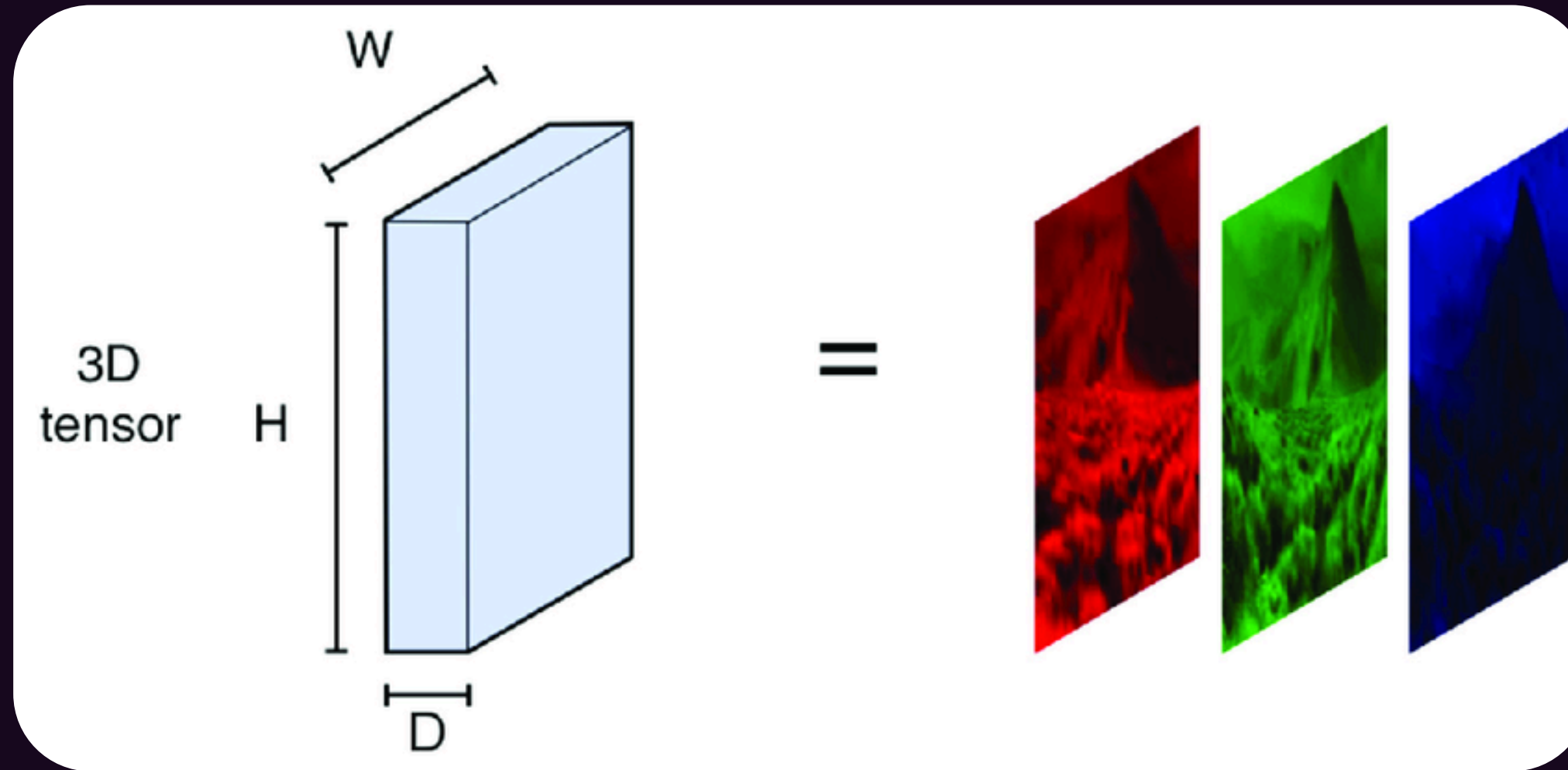


76	276	21	90	243
174	49	235	32	204
195	55	95	39	200
38	234	123	45	23
94	65	111	84	196
129	45	11	240	150
239	44	184	21	39



26	187	53	39	128
39	250	193	57	53
3	75	49	200	208
35	86	123	43	209
100	205	33	70	243
200	104	11	34	194
165	152	234	12	6

1. ¿Qué es una imagen para una red neuronal?



2. Preprocesamiento y normalización



Para que una imagen pueda ser procesada por una red neuronal, no basta con representarla como números.

1

Cambio de tamaño

Unificar dimensiones para redes con entrada fija

2

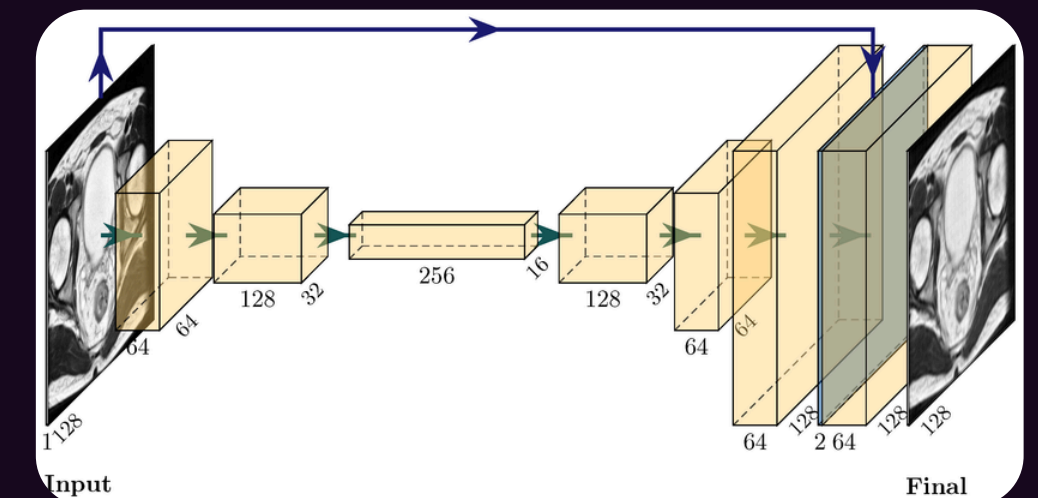
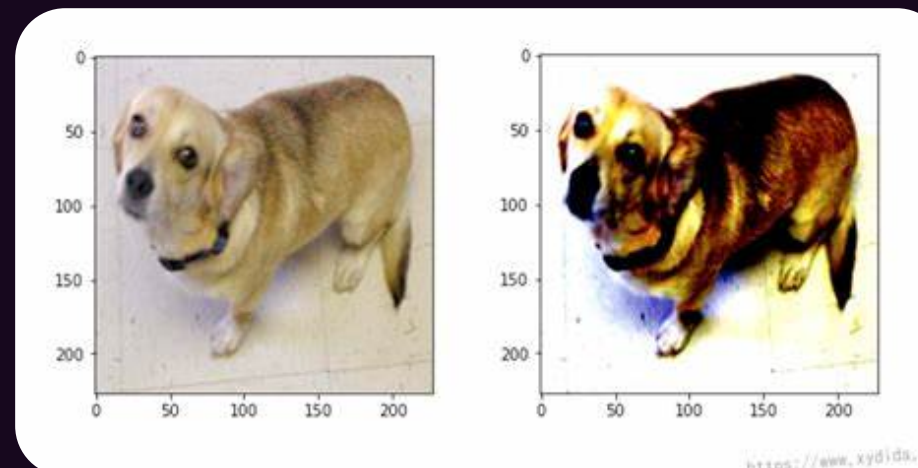
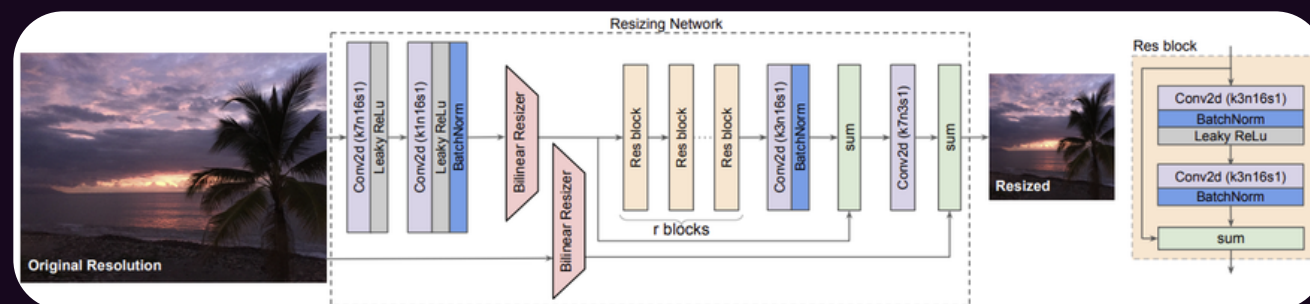
Escalado de píxeles

Acelerar el aprendizaje (por ejemplo, entre 0 y 1)

3

Normalización

Mejorar estabilidad numérica



2. Preprocesamiento y normalización

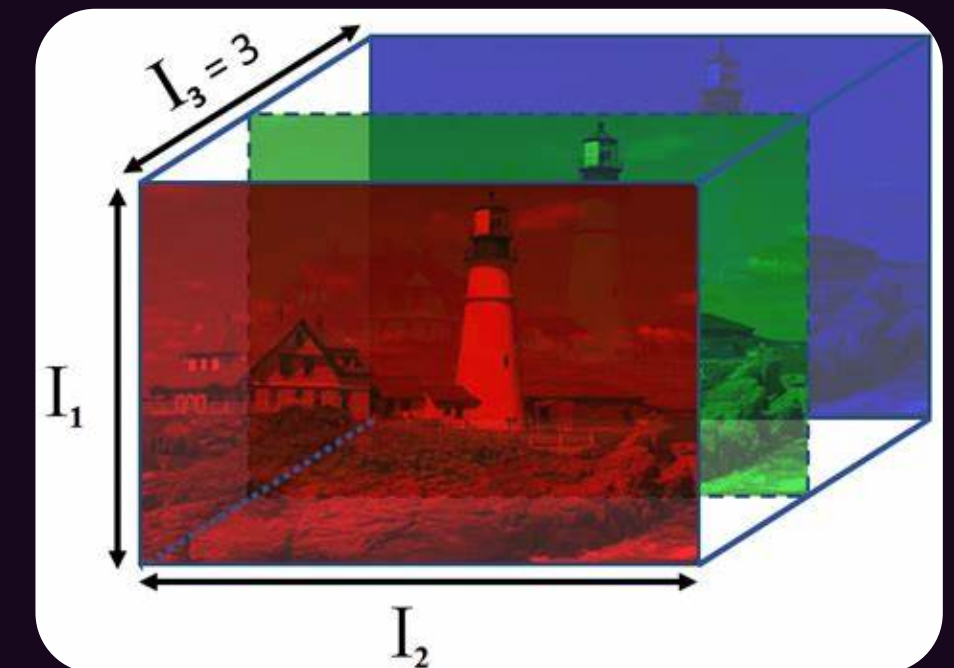
 PyTorch

4

Conversión a
tensores

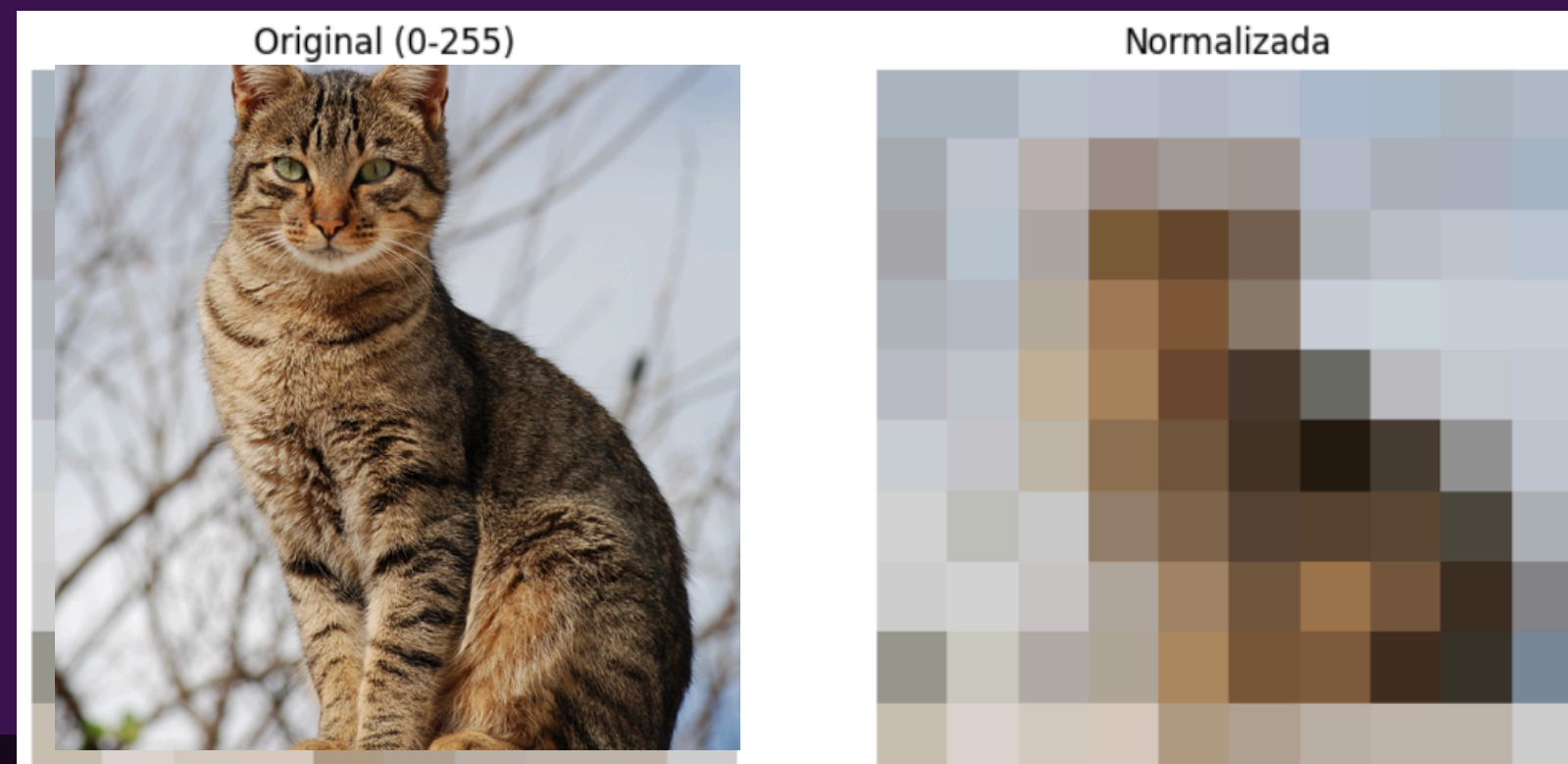
Preparar los datos para
TensorFlow, PyTorch,
etc.

Deben tener una forma clara como
(canales, alto, ancho)



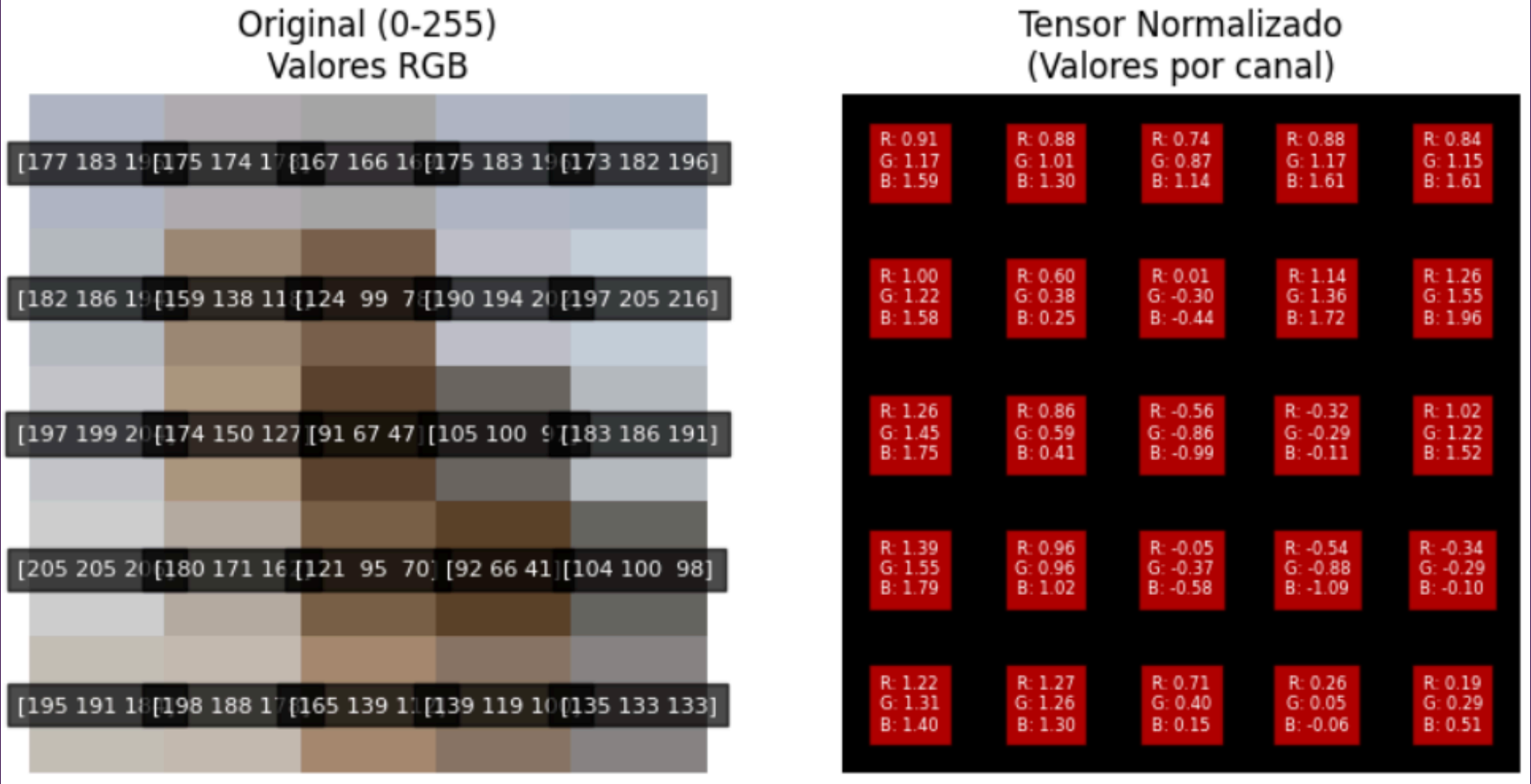
3. Ejemplo visual de transformación

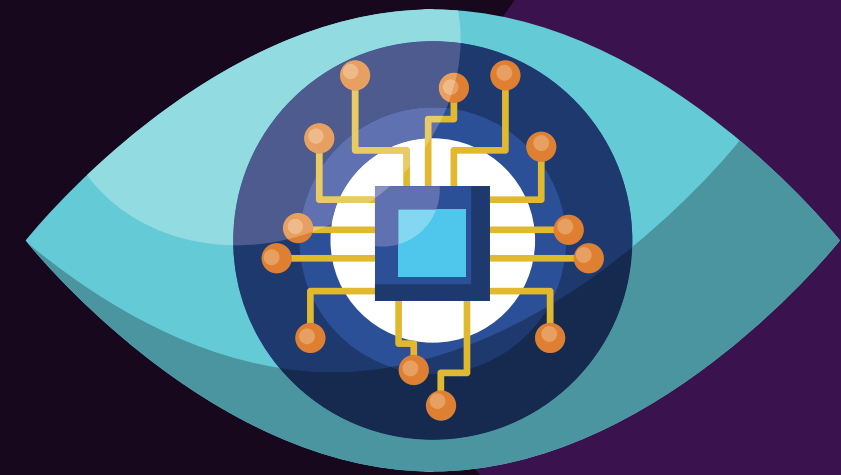
- IMAGEN PEQUEÑA (EJ: 10X10 PÍXELES) CON ZOOM EN UN PARCHE 5X5.
- *"VALORES RGB: [255, 120, 80] (0-255)"*



- MISMA IMAGEN (VISUALMENTE IDÉNTICA) CON ZOOM EN LOS VALORES DEL TENSOR.
- *"VALORES NORMALIZADOS: [0.91, -0.12, 0.45]"

TABLA DE DIFERENCIAS CLAVE

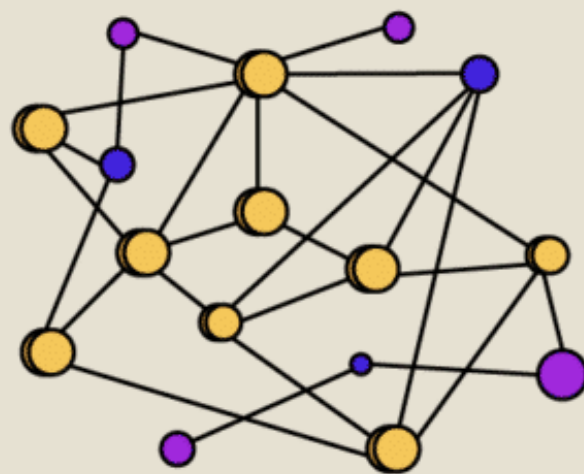




4. DIFICULTAD DE MODELAR IMÁGENES CON REDES DENSAS

¿Qué es una red densa?

Una red densa es un tipo de red neuronal donde cada neurona está conectada con todas las neuronas de la capa anterior y la siguiente. Esto funciona bien con datos tabulares o vectores pequeños, pero se vuelve inviable con imágenes.



¿Qué pasa con una imagen?

Tomemos como ejemplo una imagen de tamaño 224x224 píxeles con 3 canales RGB:

$224 \times 224 \times 3 = 150,528$ entradas

Si conectamos esta imagen a una sola capa densa con, por ejemplo, 1,000 neuronas, tendríamos:

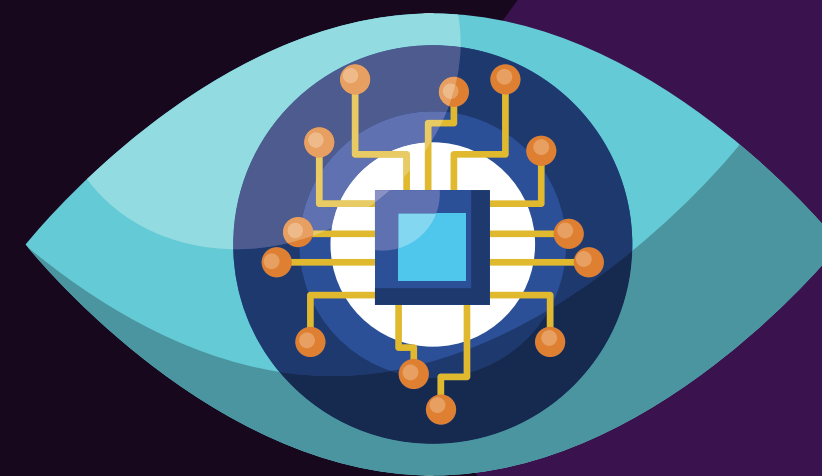
$150,528 \times 1,000 = 150,528,000$ pesos

Que son los pesos?

Los pesos son los valores numéricos que controlan la fuerza de la conexión entre dos neuronas. Son lo que la red "aprende" durante el entrenamiento.

Problemas

- Consumo de memoria: Al tener millones de pesos, el modelo es costoso de entrenar y almacenar.
- No aprovecha la estructura espacial: La red trata cada píxel como independiente. No sabe que unos píxeles están al lado de otros.
- Pierde relaciones locales: No puede captar patrones como bordes, texturas o formas.



5. VENTAJAS DE REPRESENTARLAS COMO TENSORES

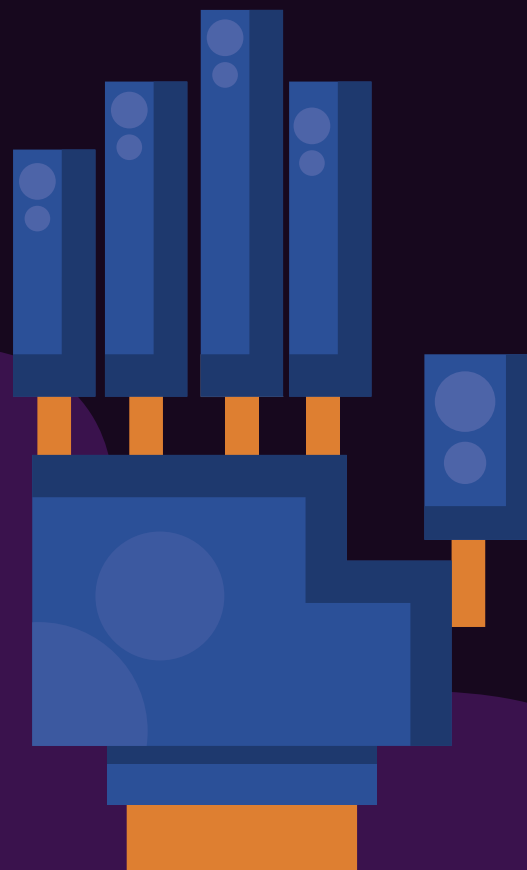


¿Qué es un tensor?

Un tensor es una estructura matemática que generaliza matrices. Una imagen RGB se representa como un tensor 3D: alto $\times$ ancho $\times$ canales.

Ejemplo:

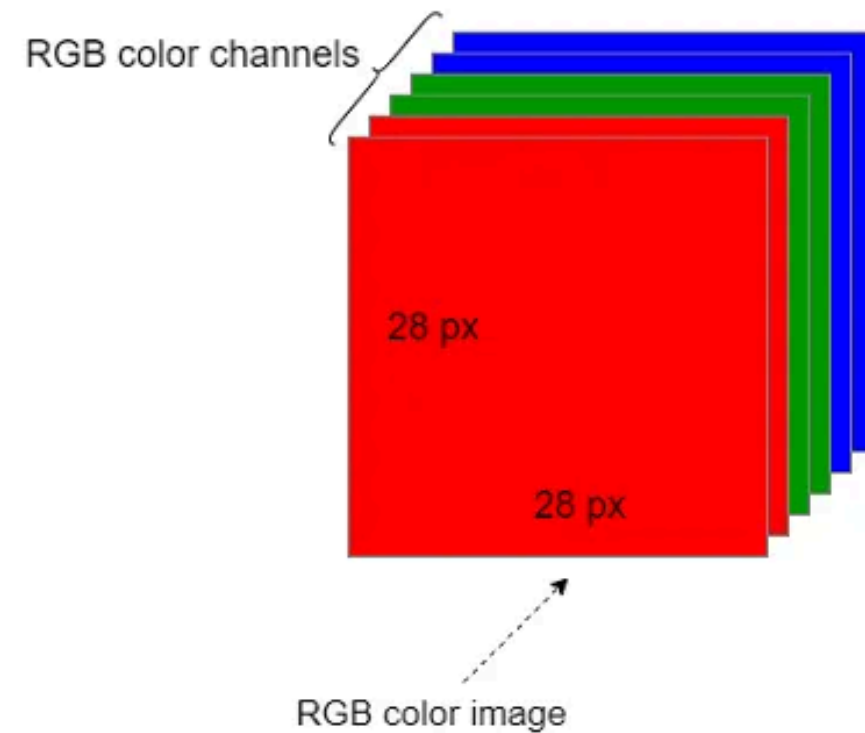
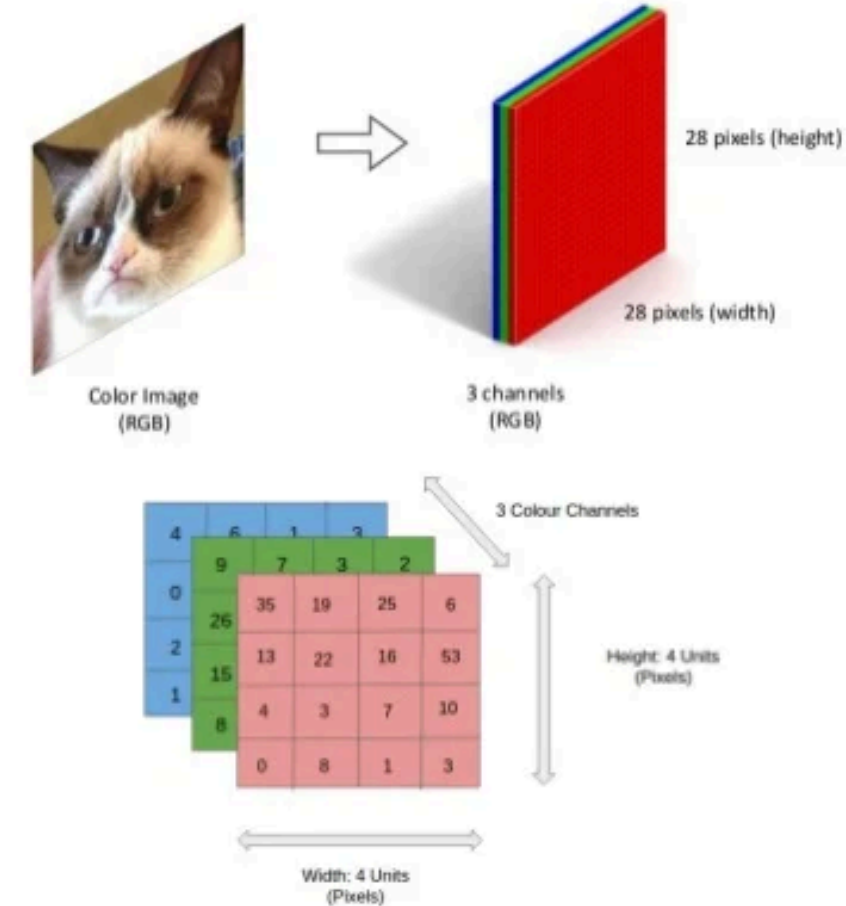
Imagen de 100 $\times$ 100 píxeles RGB=Tensor de (100,100,3)



Ventajas de usar tensores:

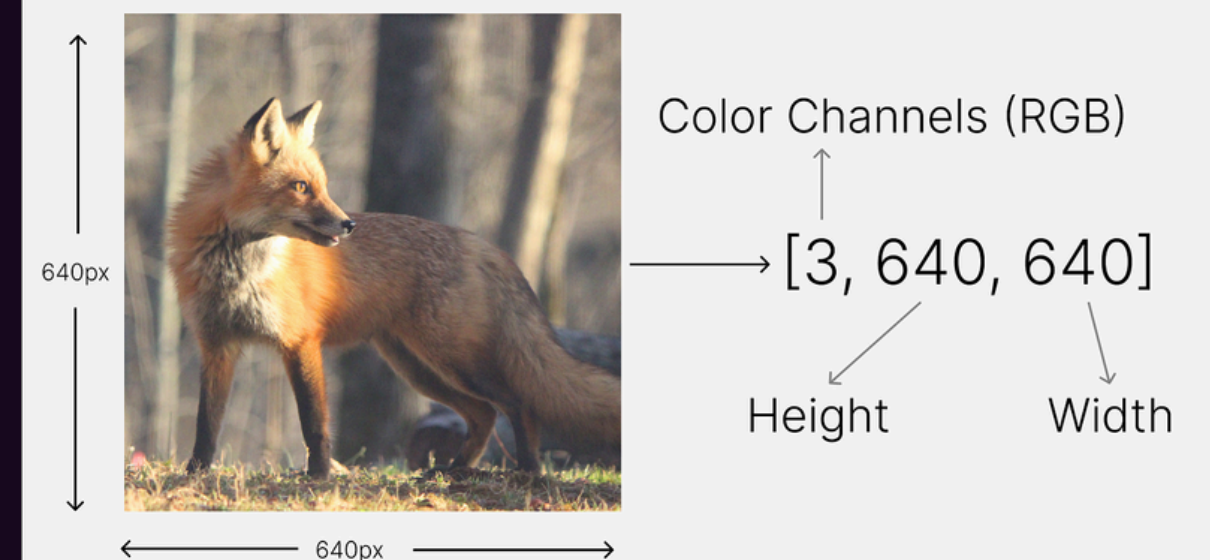
1. Preservan la estructura espacial:
 - Cada canal (R, G, B) y posición de píxel se mantiene.
 - Es posible identificar patrones locales como líneas, esquinas, texturas, etc.
2. Facilitan operaciones en paralelo:
 - Las GPUs procesan operaciones con tensores de forma eficiente (ej. convoluciones, multiplicaciones).
3. Entrada estándar para modelos de visión:
 - Clasificadores de imágenes (CNN)
 - Detectores de objetos
 - Segmentadores semánticos
4. Fundamentales para redes convolucionales:
 - Las CNN aplican filtros sobre tensores 3D conservando relaciones espaciales y contextuales.

color image is 3rd-order tensor



RGB image: An example of a rank-3 tensor (Image by author)

In this case, the three axes denote (height, width, color_channels). For example, (28, 28, 3) means that the array contains a single RGB image of size 28 x 28. The color_channels axis always takes the value 3 in the case of an RGB (Red, Green, Blue) image.



Aplicaciones típicas

En el ámbito de las aplicaciones de las redes neuronales con imágenes existen 3 grandes aplicaciones

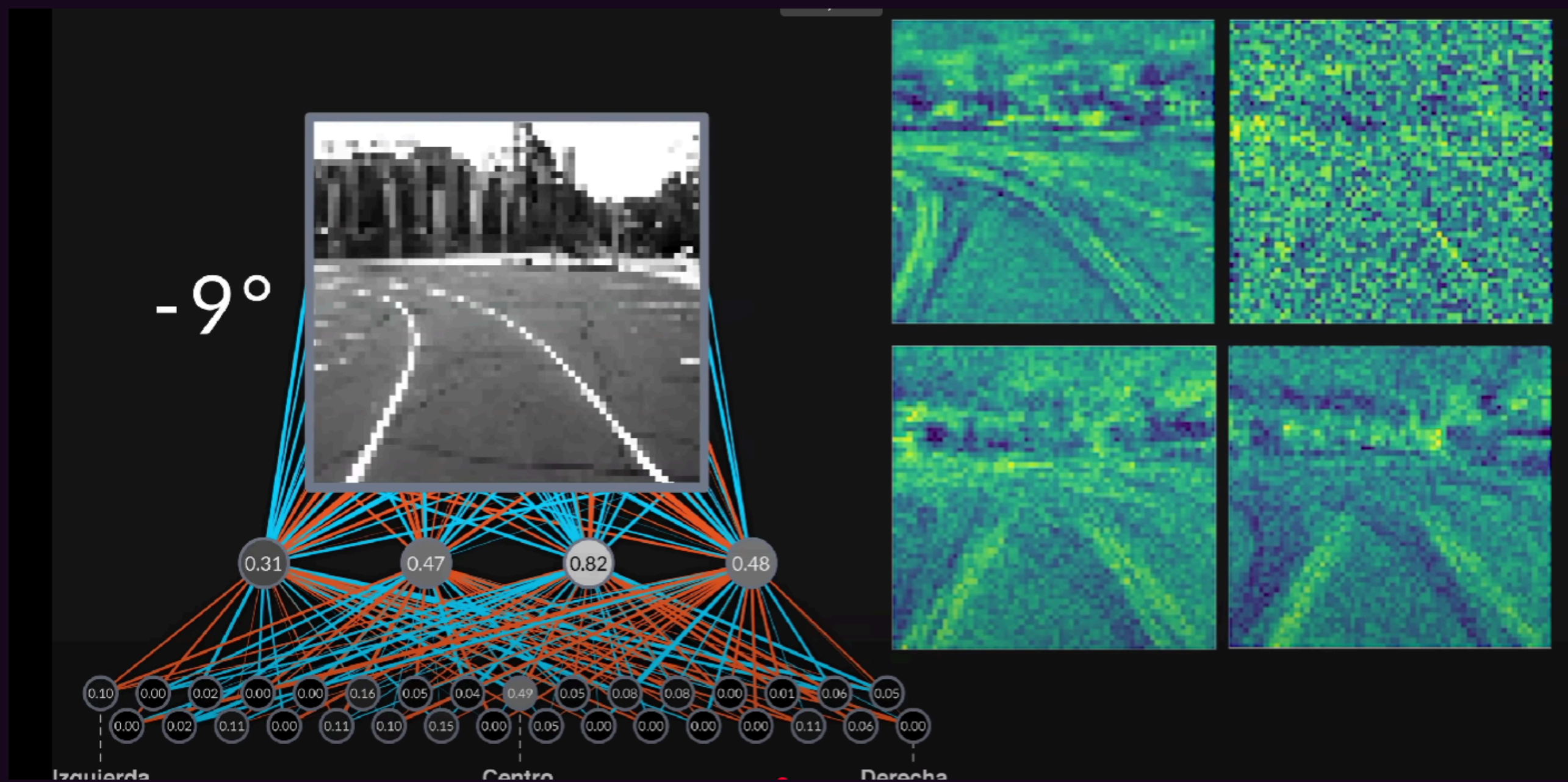


Clasificación de imágenes

Detección de bordes

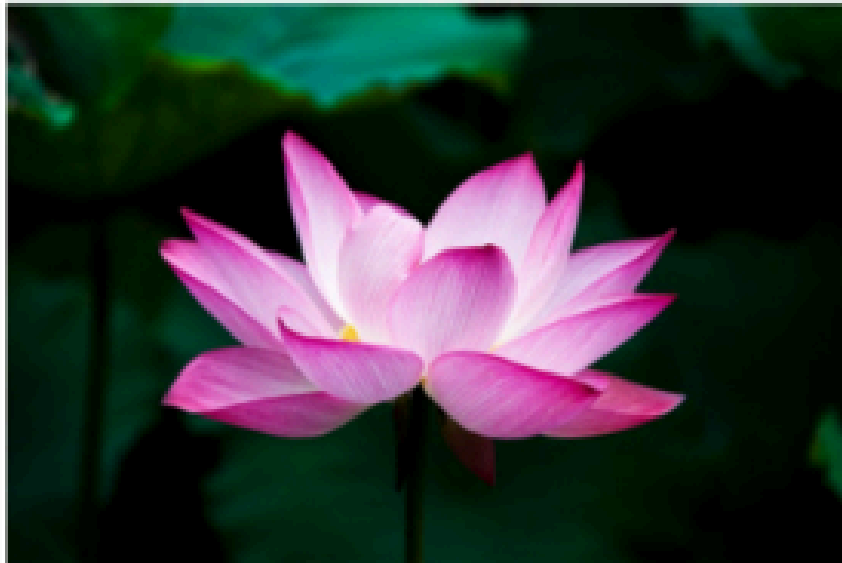
Reconocimiento de texto en imágenes

Clasificación de imágenes



Detección de bordes

Original Image



Gradient in X direction



Gradient in Y direction

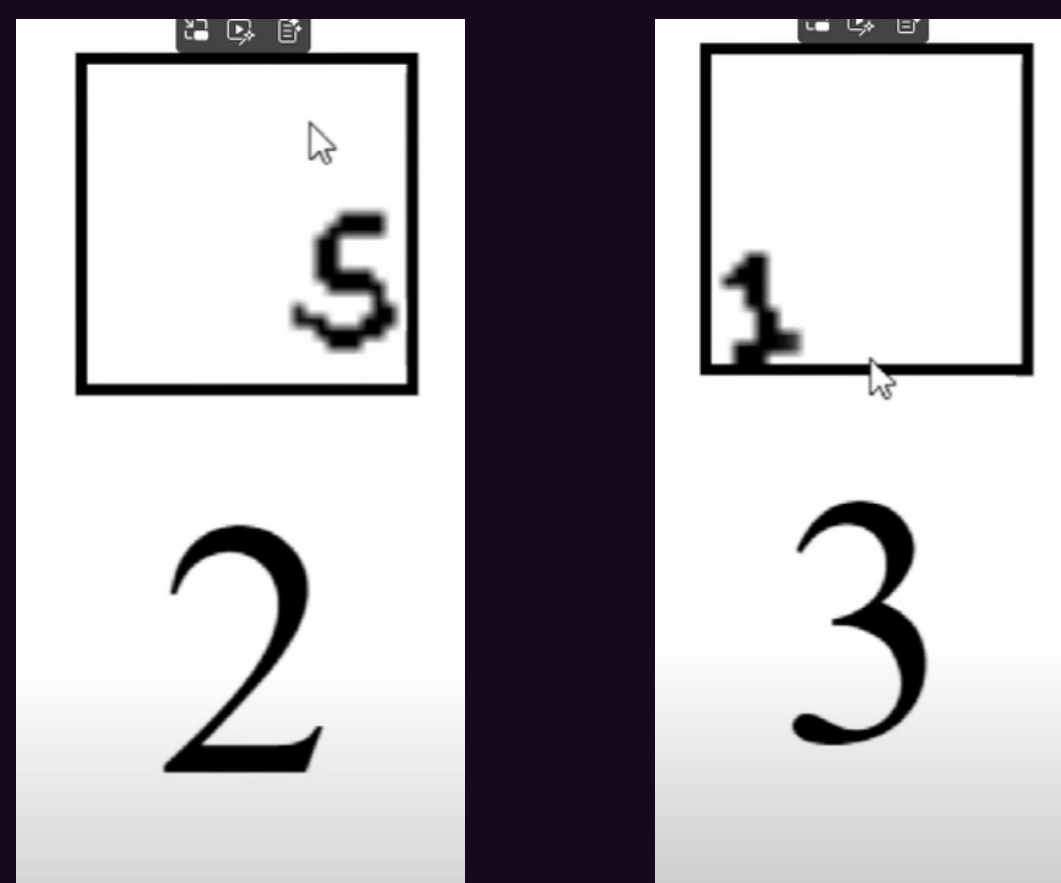


Sobel Edge Detection





Detección de bordes

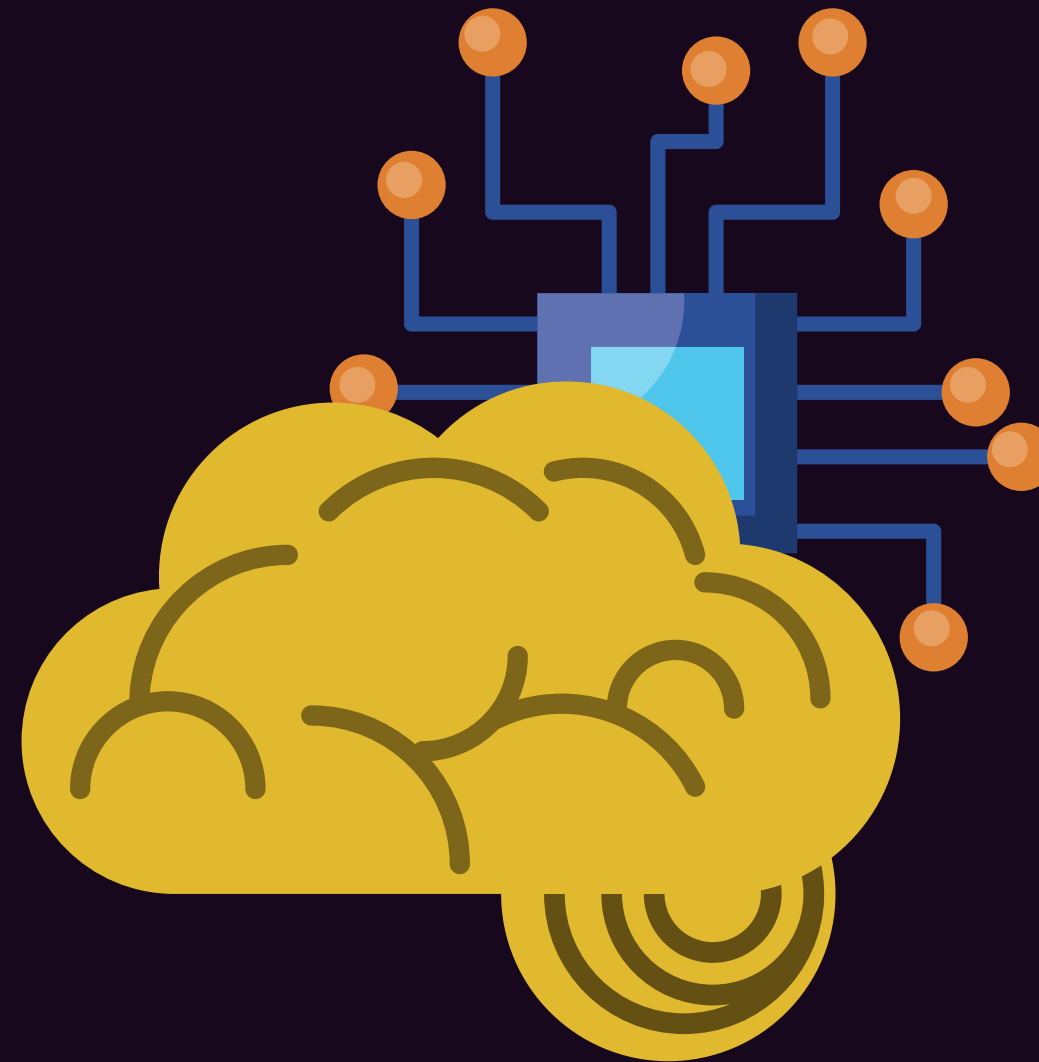


Aprendizaje

En el aprendizaje automático tradicional (por ejemplo, con modelos como SVM o regresión lineal), el procesamiento de imágenes enfrentaba un desafío fundamental: para poder utilizar estas técnicas, las imágenes (que son estructuras 2D o 3D, como píxeles o canales de color) se convertían ("aplanaban") en vectores unidimensionales.

Problema del "Aplanamiento"

- Al convertir una imagen en un vector, se pierde la estructura espacial (relaciones entre píxeles cercanos, bordes, formas, etc.).
- Por ejemplo, una imagen de 28x28 píxeles se convertía en un vector de 784 valores, ignorando que los píxeles vecinos en la imagen original podrían estar correlacionados.
- Esto limitaba el desempeño de modelos clásicos en tareas como reconocimiento de objetos o segmentación.



Ventaja de las Redes Neuronales Modernas (como CNN)

- Las redes convolucionales (CNN) y otras arquitecturas profundas preservan la estructura 2D/3D de las imágenes.
- Operan directamente sobre los píxeles organizados espacialmente, aprovechando operaciones como convoluciones (que detectan patrones locales: bordes, texturas, etc.).
- Esto permite mejor captura de características jerárquicas (desde bordes simples hasta objetos complejos) y un desempeño superior en visión por computadora.